RAG-Anything: 분석 리포트

게시용 버전: 논문 그림은 라이선스(arXiv 비독점 배포) 때문에 제외했습니다. 원문 https://arxiv.org/abs/2510.12323 · arXiv:2510.12323v1 · The University of Hong Kong (Guo, Ren, Xu, Zhang, Huang) · 코드 HKUDS/RAG-Anything (MIT, HEAD 2cd2b33 · 논문 시점 806ac9b) · 분석일 2026-10-28 · 논문 그림은 개인 학습용으로만 사용(논문 라이선스가 CC BY 아님)

핵심 요약

1. **무엇을 하는가:** MinerU로 문서를 텍스트·그림·표·수식으로 나누고, 비텍스트 단위를 VLM 설명문과 앵커 노드로 바꿔 LightRAG 지식 그래프에 넣는다. 검색된 그림은 원본 이미지로 되돌려 VLM에 준다. 사실상 LightRAG의 멀티모달 확장이다.
2. **이득은 멀티모달 질문에서 나온다:** DocBench에서 LightRAG 대비 Txt 0.0, Mm +16.6%p. 대신 답할 수 없는 질문은 MMGraphRAG보다 14.5%p 낮다. MMGraphRAG 대비 Overall +2.4%p는 1,102문항 기준 $p \approx 0.25$로 구분되지 않는다.
3. **표 정합성 문제가 있다:** MMGraphRAG의 DocBench Overall 61.0은 영역 값의 문항 가중 평균(56.7)과 4.3%p 어긋난다(나머지 행은 0.1%p 안). Table 1 평균 쪽수, Fig. 2 문항 수 합(MMLongBench 약 10% 부족), Fig. 12 질문 복사 오류도 있다.
4. **Fig. 2의 극적인 격차는 작은 구간이다:** MMLongBench 200쪽 이상의 60% vs 40%는 5문항 중 3개 vs 2개다. 긴 문서 이득의 근거는 DocBench 101쪽 이상, MMLongBench 51-200쪽이다.
5. **논문의 검색 설계 일부가 코드에 없다:** 질의어 기반 모달 선호, 융합 점수, 표 셀·그림 패널 구조 그래프는 공개 코드에 없다. 검색은 LightRAG mix 모드(키워드 벡터 검색 + 1-hop)이고, "두 그래프의 개체 정렬"은 같은 그래프에 이름이 같은 노드를 넣는 것이다.
6. **재현 스크립트의 문제:** `reproduce/query.py`는 VLM 답 생성(Eq. 6)을 끄고, 판정 fallback은 "incorrect"를 정답으로 채점한다(toy 9개 중 4개 오채점). 재현 스크립트는 논문 6개월 뒤에 추가되었다.
7. **구현 가능성:** 도구로 쓰기는 쉽다(MIT, pip 설치, API 또는 Ollama 같은 로컬 모델, CUDA 불필요). 논문 수치 재현은 비용(방법·벤치마크당 텍스트만 수십 달러)과 버전 고정 문제가 있고, 논문이 서술한 시스템 그대로는 재현할 수 없다.

01. 논문 분석: RAG-Anything (arXiv:2510.12323)

- 제목: *RAG-Anything: All-in-One RAG Framework*
- 저자: Zirui Guo, Xubin Ren, Lingrui Xu, Jiahao Zhang, Chao Huang† (The University of Hong Kong, HKUDS)
- 공개: arXiv v1 2025-10-14 (2026-10-28 기준 v1뿐) · 코드 https://github.com/HKUDS/RAG-Anything (MIT, 확인 커밋 HEAD `2cd2b33` 2026-10-28 = v1.4.4, 논문 시점 커밋 `806ac9b` 2025-10-13)
- 라이선스: arXiv 비독점 배포(nonexclusive-distrib 1.0)라 **CC BY가 아닙니다.** 논문 그림은 개인용 리포트·PPT에만 넣습니다.
- 분석 범위: 본문 12쪽과 부록 A.1-A.5(총 18쪽), 공식 코드(HEAD와 논문 시점 커밋), 기반 라이브러리 LightRAG(`aaebfe1`)
- 주제 메모: 로봇 학습 논문이 아닙니다. 멀티모달 문서 검색 증강 생성(RAG) 프레임워크입니다.

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: PDF·Office 문서를 MinerU로 텍스트, 그림, 표, 수식 단위로 쪼개고, 그림·표·수식은 VLM/LLM으로 설명문과 개체 요약을 만들어 LightRAG식 지식 그래프에 "앵커 노드"로 넣습니다. 그다음 그래프 탐색과 벡터 검색을 섞어 답합니다. 사실상 **LightRAG의 멀티모달 확장**입니다.

논문이 내세우는 기여는 세 가지입니다(Sec. 1).

1. 멀티모달 콘텐츠를 고립된 데이터가 아니라 서로 연결된 지식 개체로 다시 정의합니다(Multimodal Knowledge Unification).
2. 교차 모달 그래프와 텍스트 그래프를 따로 만든 뒤 개체 정렬로 합치는 **dual-graph construction**을 제안합니다.
3. 구조적 그래프 탐색과 의미 유사도 검색을 합친 **cross-modal hybrid retrieval**을 제안하고, 긴 문서에서 이득이 커진다고 주장합니다.

---

2. Method

2.1 전체 구조 (Fig. 1)

```
문서 k_i ──MinerU 파서──▶ 원자 단위 {c_j = (t_j, x_j)}  t_j ∈ {text, image, table, equation,
...}     (Eq. 1)
   │
   ├─ 텍스트 단위 ──▶ LightRAG 개체·관계 추출 ──▶ 텍스트 지식 그래프
   │
   └─ 비텍스트 단위 ──VLM/LLM(주변 문맥 C_j, 창 δ)──▶ 설명문 d_j^chunk + 개체 요약 e_j^entity
                        ├─ 앵커 노드 v_j^mm (이름, 유형, 요약)
                        └─ R(d_j^chunk) = (V_j, E_j) 내부 개체·관계 + (u –belongs_to→ v_j^mm)
(Eq. 2–4)
   ▼
개체 이름으로 두 그래프 병합 → G = (V, E),  임베딩 표 T = emb(개체 ∪ 관계 ∪ 청크)   → 색인 I = (G, T)
(Eq. 5)
   ▼
질의 q ──▶ (i) 구조 탐색: 개체 매칭 + 이웃 확장 → C_stru(q)
         (ii) 의미 매칭: 임베딩 top-k → C_seman(q)
         → 후보 합집합, 구조 중요도·의미 유사도·질의의 모달 선호를 합친 점수로 순위 → C*(q)
   ▼
텍스트 문맥 P(q) + 검색된 시각 자료를 원본 이미지로 되돌림 V*(q) → Response = VLM(q, P(q), V*(q))   (Eq.
6)
```

2.2 통합 표현과 dual graph (Sec. 2.2, Eq. 1-4)

- **분해 (Eq. 1):** 각 문서를 원자 단위로 나눕니다. 그림은 캡션·교차 참조와 함께, 표는 머리글·값이 있는 셀 구조로, 수식은 기호 표현(LaTeX)으로 뽑습니다.
- **교차 모달 그래프:** 비텍스트 단위마다 VLM/LLM이 설명문 두 개를 만듭니다. 검색용 상세 설명 d_j^chunk와, 그래프용 개체 요약 e_j^entity(이름, 유형, 설명)입니다. 생성할 때 주변 단위 C_j = {c_k : |k – j| ≤ δ}를 문맥으로 줍니다. 상세 설명에서 다시 개체와 관계(V_j, E_j)를 뽑고, 각 내부 개체에서 앵커 v_j^mm으로 `belongs_to` 간선을 겁니다(Eq. 3-4).
- **텍스트 그래프:** 텍스트 단위는 LightRAG·GraphRAG식으로 개체·관계를 뽑습니다.
- **병합 (Sec. 2.2.2):** 개체 이름을 기본 키로 두 그래프를 맞춰 하나의 그래프 G를 만들고, 개체·관계·청크를 모두 임베딩합니다(Eq. 5).

2.3 교차 모달 하이브리드 검색 (Sec. 2.3)

- **모달 인식 질의 인코딩:** 질의에 "figure", "chart", "table", "equation" 같은 단어가 있으면 기대 모달 신호로 씁니다. 질의는 색인과 같은 인코더로 임베딩합니다.
- **구조 탐색:** 질의어와 개체를 "정확히 매칭"한 뒤 정해진 hop 거리까지 이웃을 확장합니다.
- **의미 매칭:** 임베딩 표 T 전체에서 코사인 유사도 top-k를 뽑습니다.
- **후보 통합:** 두 경로의 후보를 합치고, 그래프 위상에서 온 구조 중요도, 의미 유사도, 질의에서 추론한 모달 선호를 합친 점수로 순위를 매깁니다. 구체적인 식이나 가중치는 없습니다.

2.4 답 생성 (Sec. 2.4, Eq. 6)

검색된 개체 요약, 관계 설명, 청크 내용을 구분자와 함께 이어 붙여 텍스트 문맥 P(q)를 만듭니다. 시각 자료에 해당하는 청크는 원본 이미지로 되돌려(V*(q)) VLM에 함께 넣습니다. 즉 검색은 텍스트 대리 표현으로 하고, 추론에는 원본 이미지를 씁니다.

3. Experiments

3.1 설정 (Sec. 3.1, Table 1, 부록 A.1)

벤치마크	문서	평균 쪽	평균 토큰	문서 유형	질문
DocBench	229	66	46,377	5 (학술, 금융, 정부, 법률, 뉴스)	1,102
MMLongBench(-Doc)	135	47.5	21,214	7	1,082

- **기준선:** GPT-4o-mini 직접 입력(문서를 최대 50쪽까지 144 dpi 이미지로 넣음), LightRAG, MMGraphRAG입니다. 모든 RAG 방법의 백본은 GPT-4o-mini입니다.
- **공통 설정:** 파서는 MinerU, 임베딩은 text-embedding-3-large(3,072차원), reranker는 bge-reranker-v2-m3입니다. 그래프 방법은 개체+관계 20,000 토큰, 청크 12,000 토큰으로 문맥을 제한하고, 답은 한 문장으로 받습니다.
- **채점:** GPT-4o-mini가 정답 대비 0/1로 판정합니다(부록 A.4, Fig. 10). 사람 평가나 다른 판정 모델은 없습니다.

3.2 주 결과 (Table 2, 3)

DocBench (Table 2, 정확도 %):

방법	Aca.	Fin.	Gov.	Law	News	Txt.	Mm.	Una.	Overall
GPT-4o-mini	40.3	46.9	60.3	59.2	61.0	61.0	43.8	49.6	51.2
LightRAG	53.8	56.2	59.5	**61.8**	65.7	**85.0**	59.7	46.8	58.4
MMGraphRAG	**64.3**	52.8	**64.9**	40.0	61.5	67.6	66.0	**60.5**	61.0
RAG-Anything	61.4	**67.0**	61.5	60.2	**66.3**	**85.0**	**76.3**	46.0	**63.4**

MMLongBench (Table 3, Overall): GPT-4o-mini 33.5, LightRAG 38.9, MMGraphRAG 37.7, RAG-Anything **42.8**입니다. 7개 영역 중 4개에서 최고입니다.

해석 (재계산은 `04_code_verification.md` §3과 `analysis/table_check.py`):

- **MMGraphRAG의 DocBench Overall 61.0은 자기 영역 값과 맞지 않습니다.** 다섯 영역 정확도를 Table 5의 문항 수(303, 288, 148, 191, 172)로 가중 평균하면 56.7입니다. 다른 세 방법과 Table 4 절제 세 행은 모두 0.05%p 안에서 맞습니다. 칸 하나의 오기라면 Law 40.0 → 64.7 또는 Fin 52.8 → 69.2일 때 61.0이 됩니다. Fig. 2의 구간별 값으로 가중 평균하면 60.9라서, Overall보다는 영역 칸 하나가 잘못되었을 가능성이 큽니다(가설). 어느 쪽이든 "MMGraphRAG가 2위"라는 순위 해석은 이 칸에 달려 있습니다.
- **이득은 대부분 Mm 유형에서 나옵니다.** LightRAG 대비 차이는 Txt 0.0, Mm +16.6, Una −0.8%p입니다. 텍스트만 필요한 질문에서는 기반 라이브러리와 똑같고, 멀티모달 질문에서 이깁니다. 설계 의도와 맞는 결과입니다.
- **답할 수 없는 질문(Una.)에서는 MMGraphRAG보다 14.5%p 낮습니다**(46.0 vs 60.5). 근거가 없을 때 "답할 수 없다"고 말하는 능력은 떨어집니다. 본문은 이 열을 논의하지 않습니다.
- **MMGraphRAG 대비 DocBench 차이는 통계적으로 구분되지 않습니다.** +2.4%p이고 1,102문항 비대응 근사로 $p \approx 0.25$입니다. LightRAG 대비 +5.0%p($p \approx 0.016$), MMLongBench에서는 MMGraphRAG 대비 +5.1%p($p \approx 0.016$), LightRAG 대비 +3.9%p($p \approx 0.065$)입니다. 같은 문항을 푼 결과라 대응 검정을 하면 더 민감하지만, 문항별 결과가 공개되지 않았습니다.
- **GPT-4o-mini 기준선은 구조적으로 불리합니다.** 50쪽까지만 이미지로 넣으므로, DocBench 금융 문서(평균 192쪽)는 약 26%만 봅니다. "긴 문서에서 전통적 방법이 실패한다"는 문장의 일부는 이 잘림에서 옵니다.

3.3 문서 길이별 성능 (Fig. 2)

그림을 다시 렌더링해 픽셀로 읽었습니다(정확도 ±0.3%p, 문항 수 ±2).

구간(쪽)	DocBench n	RAG-Anything	MMGraphRAG	MMLongBench n	RAG-Anything	MMGraphRAG
1-10	≈339	61.9	64.8	**≈16**	18.6	12.3
11-50	≈348	≈61.6	62.0	≈705	43.7	40.3
51-100	≈160	61.9	59.5	≈148	39.6	30.4
101-200	≈153	68.1	54.5	≈100	41.5	33.6
200+	≈79	68.7	54.9	**≈5**	59.9	39.9

- 본문의 길이별 수치(68.2 vs 54.6, 68.8 vs 55.0, MMLongBench +3.4/+9.3/+7.9%p)는 그림 판독값과 0.1%p 안에서 맞습니다.
- DocBench의 장점은 **101쪽 이상에서만** 나타납니다(+13.6, +13.8%p). 50쪽 이하에서는 MMGraphRAG와 같거나 낮습니다.
- **MMLongBench 양끝 구간은 문항이 거의 없습니다.** 200쪽 이상 구간의 60% vs 40%는 5문항 중 3개 vs 2개, 즉 한 문항 차이입니다(95% 구간 [23, 88] vs [12, 77]). 1-10쪽 구간은 16문항 중 3개 vs 2개입니다. 그림에서 가장 눈에 띄는 200쪽 이상의 격차는 근거가 되지 못합니다.
- 그림의 문항 수를 더하면 DocBench 약 1,079(표 1,102), MMLongBench 약 971(표 1,082)입니다. MMLongBench는 약 10%가 그림에 없습니다. 판독 오차(±2/막대)로는 설명되지 않습니다.

3.4 절제 실험 (Table 4)

변형	Overall	전체 대비
Chunk-only (그래프 없이 청크 검색만)	60.0	−3.4 (p ≈ 0.10)
w/o Reranker	62.4	−1.0 (p ≈ 0.63)
RAG-Anything	63.4	

- 저자들은 "그래프 구성이 필수"라고 결론짓습니다. 하지만 근거는 3.4%p 차이 하나이고, 비대응 근사로는 유의하지 않습니다.
- dual graph의 두 구성 요소(교차 모달 그래프만 / 텍스트 그래프만), 모달 인식 질의 인코딩, VLM 답 생성(Eq. 6)을 각각 끄는 절제는 없습니다. 논문이 내세운 두 핵심 기여 중 어느 쪽이 이득을 냈는지 분리되지 않습니다.

3.5 사례 연구 (Sec. 3.4, 부록 A.2, A.5)

- 성공 사례 4개(다중 패널 그림, 금융 표, 막대그래프, 반복 행 이름이 있는 표)에서, 저자들은 RAG-Anything이 표를 "행 머리글, 열 머리글, 셀, 단위 노드와 row-of, column-of, header-applies-to, unit-of 간선"으로, 그림을 "패널, 축 제목, 범례, 캡션 노드"로 만든다고 설명합니다. 이런 명시적 구조는 **공개 코드에 없습니다**(`04` D2).
- 실패 사례 2개(A.5)는 모든 방법이 틀린 경우입니다. 텍스트 쪽으로 치우친 검색, 그리고 병합 셀이 있는 표 파싱 실패입니다. 이 실패 분석은 솔직하고 유용합니다.
- **Fig. 12의 질문 문구가 Fig. 3의 질문을 그대로 복사한 것입니다**("Which model's style space shows a clearer separation…"). 답변들은 "Train" 영역의 Joint goal accuracy를 이야기하므로 그림 편집 오류로 보입니다.

---

4. Limitations

4.1 저자가 밝힌 한계 (Sec. 5, 부록 A.5)

텍스트 중심 검색 편향(질의가 그림을 요구해도 텍스트 근거를 먼저 가져옴)과, 비표준 레이아웃·병합 셀 표에서의 경직된 공간 처리입니다. 적응형 공간 추론과 레이아웃 인식 파싱이 필요하다고 정리합니다.

4.2 분석하면서 추가로 발견한 한계와 의문점

1. **논문이 설명하는 검색 장치 여러 개가 코드에 없습니다**(`04` D3, D4). 질의어 기반 모달 선호, 구조 중요도·의미·모달을 합친 융합 점수, "정확한 개체 매칭 + 지정 hop 확장"이 그것입니다. 코드의 검색은 LightRAG `mix` 모드입니다. LLM이 뽑은 키워드로 개체·관계 벡터 DB를 검색하고(top_k, 코사인 ≥ 0.2), 1-hop 이웃을 붙이고, 벡터 청크와 합칩니다. 논문은 이 장치들을 하나의 새로운 하이브리드 검색처럼 서술하지만, 구현은 기반 라이브러리의 기존 모드입니다.
2. **"두 그래프를 개체 정렬로 병합"은 같은 그래프 저장소에 이름이 같은 노드를 넣는 것입니다**(`04` D1). 앵커 노드 이름에는 " (image)" 같은 유형 접미사가 붙어서 텍스트 개체와는 병합되지 않습니다. 두 그래프는 내부 개체 이름이 문자열로 정확히 같을 때만 이어집니다. 동의어나 표기 변형을 맞추는 정렬 단계는 없습니다.
3. **공개된 재현 스크립트는 논문 식 6을 실행하지 않습니다**(`04` D5). `reproduce/query.py`는 `vlm_enhanced=False`로 질의하므로 답은 텍스트 문맥만 보고 생성됩니다. 재현 스크립트는 논문보다 6개월 늦은 2026-04-13에 추가되었습니다. 논문 수치가 VLM 답 생성으로 얻은 것인지는 알 수 없습니다.
4. **판정 스크립트에 채점 버그가 있습니다**(`04` D8). JSON 파싱에 실패하면 응답에서 키워드를 찾는데, 긍정 키워드를 부분 문자열로 먼저 검사합니다. 그래서 "incorrect", "mismatch", "not accurate"가 정답(1)으로 채점됩니다. toy 응답 9개 중 4개가 잘못 채점되었습니다. 실제 영향은 파싱 실패 빈도에 달려 있고, 그 빈도는 공개되지 않았습니다.
5. **판정자와 백본이 같은 모델입니다.** 모든 방법의 생성 모델과 채점 모델이 GPT-4o-mini입니다. 자기 문체에 관대한 편향이 있을 수 있고, 사람 평가로 판정자를 검증하지 않았습니다. MMLongBench-Doc의 원래 채점 방식(답 추출 후 규칙 기반 비교)과도 달라서, 공개 리더보드 수치와 직접 비교할 수 없습니다(원 벤치마크 채점 방식은 내 사전 지식, 확인 필요).
6. **표 수치의 정합성 문제가 여러 개입니다.** MMGraphRAG Overall(위 §3.2), Table 1 평균 쪽수(DocBench 66 vs Table 5에서 다시 계산한 61.0, MMLongBench 47.5 vs 48.3), Fig. 2 문항 수 합(MMLongBench 약 10% 부족), Fig. 12 질문 복사 오류입니다. 하나하나는 작지만, 결과 표를 직접 검산할 필요가 있다는 신호입니다.
7. **시드, 분산, 비용이 없습니다.** LLM 호출이 확률적인데 반복 실행이 없고, 색인·질의 비용과 시간이 보고되지 않았습니다. 이 방식은 비텍스트 단위마다 VLM 호출이 추가되므로, 텍스트 RAG보다 색인 비용이 큽니다.
8. **그래프 구성의 이득과 비용이 비교되지 않았습니다.** Chunk-only 대비 +3.4%p를 얻으려고 청크마다 LLM 추출(gleaning 포함 2회)을 돌립니다. 같은 비용을 더 강한 reader나 더 긴 문맥에 썼을 때와 비교한 실험은 없습니다.

03. 수치 재계산과 해석

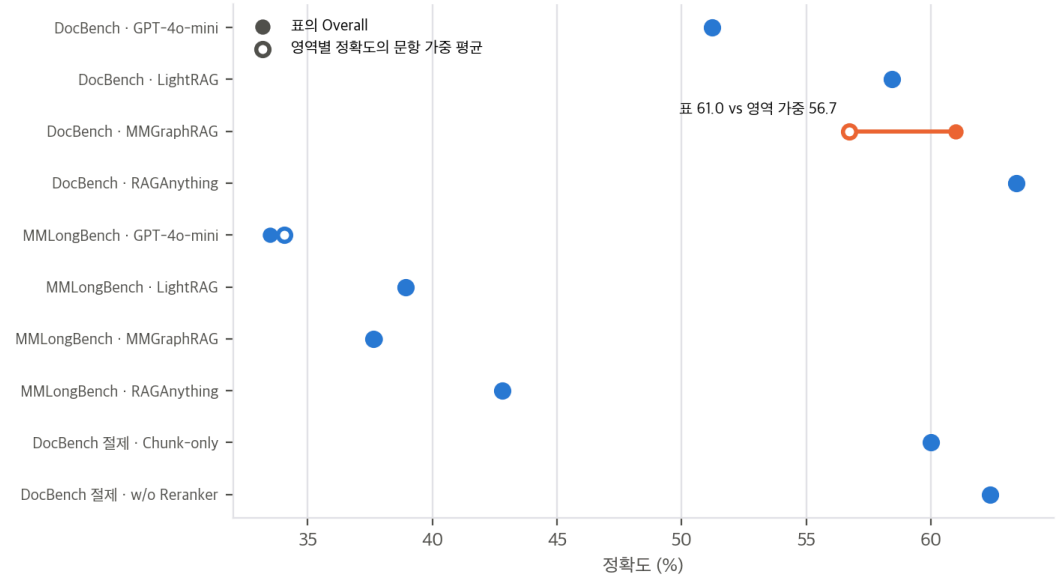
재계산 스크립트는 `analysis/table_check.py`(표·그림 수치, 결과 `table_check_results.json`)와 `analysis/code_logic_toy.py`(공식 코드 두 곳의 동작, 결과 `code_logic_toy_results.json`)입니다. 외부 API를 부르지 않고 CPU에서 몇 초 안에 끝납니다. 그래프는 `analysis/plot_figures.py`로 그렸습니다.

1. Overall은 영역 값의 문항 가중 평균이어야 한다 (C1)

DocBench의 Overall은 문항 1,102개의 정답률이므로, 영역별 정확도 a_d를 영역 문항 수 n_d(Table 5: 303, 288, 148, 191, 172)로 가중 평균한 값과 같아야 합니다.

```
Overall = Σ_d n_d · a_d / Σ_d n_d      (반올림 오차는 ±0.05%p 이내)
```

행	표의 Overall	영역 가중 평균	차이
GPT-4o-mini (T2)	51.2	51.22	0.0
LightRAG (T2)	58.4	58.44	0.0
MMGraphRAG (T2)	**61.0**	**56.73**	**+4.3**
RAG-Anything (T2)	63.4	63.43	0.0
Chunk-only / w/o Reranker (T4)	60.0 / 62.4	60.00 / 62.38	0.0
MMLongBench 네 방법 (T3)	33.5 / 38.9 / 37.7 / 42.8	34.05 / 38.92 / 37.64 / 42.81	GPT-4o-mini만 −0.55



출처: Table 2·3·4와 문항 수 Table 5·6. analysis/table_check.py C1

MMGraphRAG 행만 4.3%p 어긋납니다. 칸 하나가 잘못 적혔다고 가정하고, Overall 61.0을 맞추려면 각 영역 값이 얼마여야 하는지 역산했습니다.

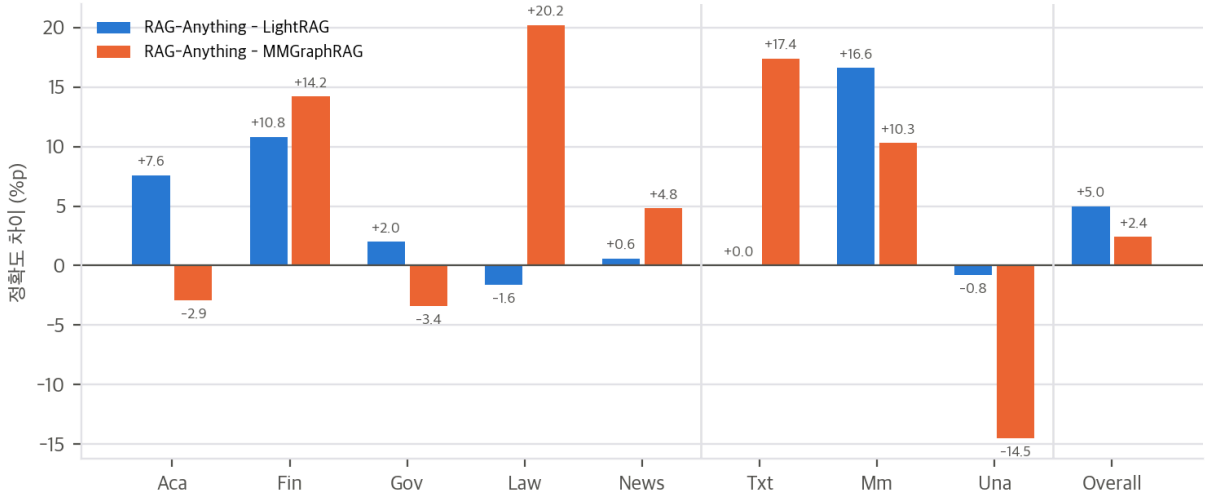
칸	표의 값	61.0이 되려면	그럴듯한가
Aca	64.3	79.8	낮음
Fin	52.8	69.2	가능 (그러면 Fin 1위가 MMGraphRAG)
Gov	64.9	96.7	아님

칸	표의 값	61.0이 되려면	그럴듯한가
Law	**40.0**	**64.7**	**가능** (다른 방법의 Law는 59-62. 40.0이 유일한 이상값)
News	61.5	88.9	아님

Fig. 2의 구간별 정확도를 그림의 문항 수로 가중 평균하면 MMGraphRAG는 60.9입니다(§3). Overall 61.0이 실제 계산값이고, 영역 칸 하나(Law나 Fin)가 잘못 적혔을 가능성이 큽니다. 이것은 가설입니다. Law가 64.7이었다면 RAG-Anything은 Law에서 2위가 아니라 3위이고, Fin이 69.2였다면 Fin 1위를 놓칩니다.

2. 이득은 어디서 오는가: 유형 분해 (C2, C7)

차이 (%p)	Txt.	Mm.	Una.	Overall
RAG-Anything – LightRAG	0.0	**+16.6**	–0.8	+5.0
RAG-Anything – MMGraphRAG	+17.4	+10.3	**–14.5**	+2.4



출처: Table 2 (DocBench, GPT-4o-mini 판정). 영역 | 유형 | Overall. MMGraphRAG Overall 61.0은 영역 가중 평균 56.7과 불일치(C1)

- **LightRAG 대비 이득은 전부 멀티모달 질문에서 나옵니다.** Txt는 85.0으로 같습니다. RAG-Anything이 텍스트 처리에서는 LightRAG를 그대로 쓴다는 코드 구조(`04` §1)와 정확히 맞습니다.
- **답할 수 없는 질문에서는 MMGraphRAG보다 크게 낮습니다.** 검색이 많은 근거를 가져오면 reader가 근거 없는 질문에도 답을 만들어 내는 경향으로 읽을 수 있습니다. 본문은 이 열을 다루지 않습니다.
- **유형 세 개만으로는 Overall을 맞출 수 없습니다.** Table 2·4의 7개 행에 공통 유형 비율(Txt, Mm, Una)을 비음수 최소제곱으로 맞추면 잔차가 최대 2.7%p(MMGraphRAG)입니다. DocBench 원 논문에는 메타데이터 질문 유형도 있는 것으로 아는데, 표에 없는 유형이 Overall에 섞여 있다면 설명됩니다. 원 벤치마크 유형 구성은 내 사전 지식이라 확인이 필요합니다. 그래서 이 맞춤에서 나온 유형별 문항 수 추정(259, 305, 538)은 믿을 만하지 않습니다.

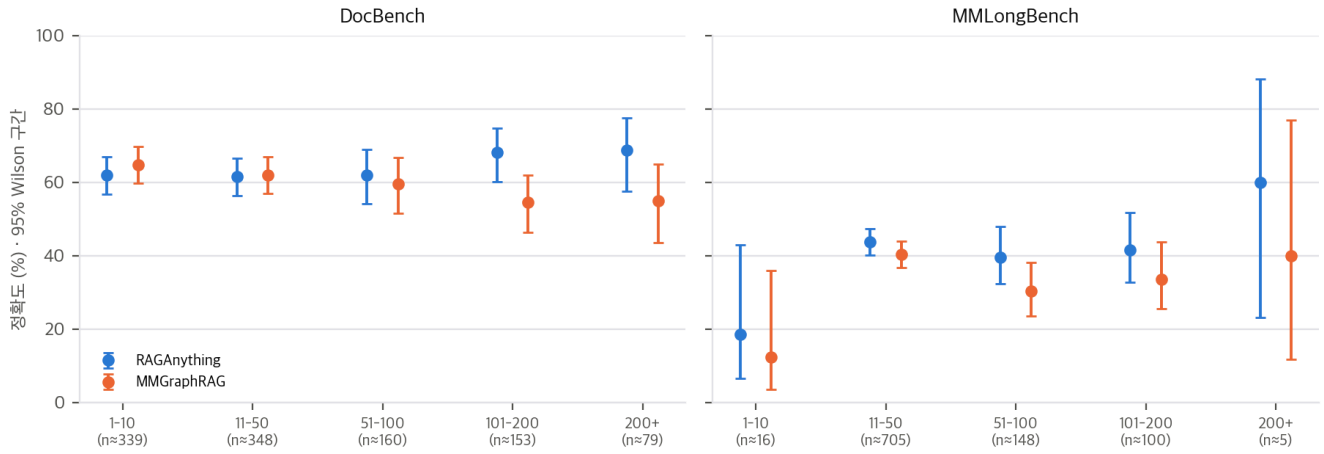
3. Fig. 2 길이 구간: 작은 구간은 한두 문항 차이다 (C5)

Fig. 2를 300 dpi로 다시 렌더링하고, 막대 위끝과 표식 중심을 픽셀로 읽었습니다. 정확도 축은 64 px가 20%p, 문항 수 축은 DocBench 0.874 px/문항, MMLongBench 0.431 px/문항입니다.

- **본문 수치와 맞습니다.** 101-200쪽 68.1 vs 54.5(본문 68.2 vs 54.6), 200쪽 이상 68.7 vs 54.9(본문 68.8 vs 55.0), MMLongBench 차이 +3.4, +9.2, +7.9(본문 +3.4, +9.3, +7.9)입니다.

- **Overall도 재현됩니다.** 구간 정확도를 그림의 문항 수로 가중 평균하면 DocBench 63.2 / 60.9, MMLongBench 42.6 / 37.7입니다(표 63.4 / 61.0, 42.8 / 37.7).
- **MMLongBench 양끝 구간은 문항 수가 작습니다.** 막대가 6 px, 2 px 이하라 높이로는 셀 수 없어서, 두 방법의 정확도가 동시에 k/n이 되는 가장 작은 n을 찾았습니다.

구간	n	RAG-Anything	MMGraphRAG	Fisher p
1-10쪽	16	3/16 = 18.8%, 95% [6.6, 43.0]	2/16 = 12.5%, [3.5, 36.0]	1.0
200쪽 이상	5	3/5 = 60%, [23.1, 88.2]	2/5 = 40%, [11.8, 76.9]	1.0



출처: Fig. 2를 다시 렌더링해 픽셀로 판독(정확도 ±0.3%p, 문항 수 ±2). MMLongBench 1-10·200+ 구간 n은 정확도가 k/n이 되는 값

그림에서 가장 극적으로 보이는 MMLongBench 200쪽 이상의 60% vs 40%는 한 문항 차이입니다. 긴 문서에서의 이득은 DocBench 101쪽 이상(약 230문항)과 MMLongBench 51-200쪽(약 250문항)에서만 근거가 있습니다.

- **그림 문항 수 합이 표와 다릅니다.** DocBench 약 1,079(표 1,102, −2%), MMLongBench 약 971(표 1,082, −10%)입니다. 막대 하나당 판독 오차 ±2로는 MMLongBench의 110문항 차이를 설명할 수 없습니다. 쪽수 정보가 없는 문서가 빠졌거나 다른 부분집합으로 그렸을 수 있습니다.

4. 차이의 표본 오차 (C3)

문항별 결과가 공개되지 않아서, 같은 n의 두 비율을 비대응 z 검정으로 근사했습니다. 같은 문항을 푼 대응 자료이므로 실제 McNemar 검정은 이보다 민감할 수 있습니다. 아래 p값은 보수적인 근사로 읽어야 합니다.

비교	차이	p (비대응 근사)
DocBench vs MMGraphRAG	+2.4	0.25
DocBench vs LightRAG	+5.0	0.016
MMLongBench vs MMGraphRAG	+5.1	0.016
MMLongBench vs LightRAG	+3.9	0.065
Chunk-only → 전체 (Table 4)	+3.4	0.10
w/o Reranker → 전체	+1.0	0.63

RAG-Anything의 Overall 95% Wilson 구간은 DocBench [60.5, 66.2], MMLongBench [39.9, 45.8]입니다. "그래프 구성이 필수"(Sec. 3.3)라는 결론은 p ≈ 0.10인 3.4%p 차이 하나에 기대고 있습니다.

5. 코드가 만드는 그래프의 실제 모양 (G)

`modalprocessors.py`를 읽으면 그래프 구성 규칙은 다음과 같습니다(논문 시점 `806ac9b`와 HEAD 모두 같음).

1. 비텍스트 단위마다 앵커 노드를 만듭니다. 이름은 VLM이 준 이름에 `" (image)"`, `" (table)"` 같은 유형 접미사를 붙인 것입니다(HEAD 1139-1141행).
2. 상세 설명문을 하나의 청크로 저장하고, LightRAG의 일반 개체 추출(`extract_entities`)을 돌립니다.
3. 추출된 내부 개체마다 `개체 → 앵커`로 `belongs_to` 간선(weight 10.0)을 추가합니다.
4. 텍스트 쪽과 같은 그래프 저장소에 넣으므로, 이름 문자열이 정확히 같은 노드는 자동으로 합쳐집니다.

그래서 텍스트 쪽 개체에서 출발한 1-hop 탐색이 그림 앵커에 닿으려면, 앵커의 내부 개체 m개 중 하나 이상이 텍스트 개체와 이름이 정확히 같아야 합니다. 내부 개체 하나가 텍스트 개체와 같은 이름일 확률을 p라고 두면 다음과 같습니다.

```
P(앵커가 텍스트 그래프와 연결) = 1 − (1 − p)^m
```

p \ m	2	5	10
0.05	0.10	0.23	0.40
0.1	0.19	0.41	0.65
0.2	0.36	0.67	0.89
0.4	0.64	0.92	0.99

(시뮬레이션 2,000개 앵커로 확인, `code_logic_toy.py` G)

해석:

- p는 "같은 개념을 그림 설명과 본문에서 LLM이 똑같은 문자열로 부를 확률"입니다. 동의어나 대소문자, 단·복수 표기 차이가 있으면 떨어집니다. 실제 값은 데이터에 따라 다르고 이 toy로는 알 수 없습니다.
- 연결되지 않은 앵커도 버려지지는 않습니다. LightRAG의 개체 검색은 키워드 임베딩으로 개체 벡터 DB를 찾으므로(`operate.py _get_node_data`), 앵커나 내부 개체가 직접 벡터로 검색될 수 있습니다. 즉 이 시스템에서 그림에 닿는 주된 경로는 "그래프 탐색"보다 **개체 설명의 벡터 검색**일 가능성이 큽니다. 논문이 강조하는 "dual graph의 개체 정렬"이 실제로 얼마나 기여하는지 보려면, 병합을 끈 절제가 필요합니다.

6. 판정 fallback의 부분 문자열 버그 (E)

`reproduce/llm_answer_evaluator.py`의 `create_fallback_evaluation`(229-268행)은 판정 응답이 JSON으로 파싱되지 않을 때 쓰입니다. 순서는 다음과 같습니다.

1. `"accuracy": n`, `accuracy … n`, `correct … n`, `accurate … n` 패턴에서 첫 숫자를 찾습니다. 그 숫자가 0이나 1이 아니면 그 패턴은 버립니다.
2. 그래도 없으면 긍정 키워드(`correct`, `accurate`, `right`, `yes`, `true`, `match`)를 **부분 문자열로 먼저** 검사하고, 그다음 부정 키워드를 봅니다.

규칙을 그대로 옮겨 9개 응답에 적용한 결과입니다.

판정 응답	의도	채점
"The generated answer is incorrect."	0	1

판정 응답	의도	채점
"Incorrect: the expected value is DKK 26,778 million, …"	0	**1** (첫 숫자 26이 0/1이 아니라 패턴을 버리고, "incorrect" ⊃ "correct")
"… does not match … (mismatch in the year)."	0	**1**
"Not accurate; the model chose the VAE panel."	0	**1**
"This is wrong."	0	0
"The generated answer is correct."	1	1
닫는 괄호가 없는 `{"accuracy": 1, …` / `{"accuracy": 0, …`	1 / 0	1 / 0
"Accuracy: 0 because the panel is wrong"	0	0

의도가 0인 응답 7개 중 4개가 정답으로 채점됩니다. 기본 판정 모드가 여러 지표를 함께 요구하는 `comprehensive`라서 JSON이 길어지고 깨질 여지도 커집니다. 다만 이 경로는 파싱이 실패할 때만 쓰이므로, 논문 수치에 실제로 얼마나 영향을 주었는지는 실패 빈도를 알아야 판단할 수 있습니다. 이 스크립트는 논문 이후에 추가되었으므로, 논문 수치를 이 스크립트로 냈다는 증거도 없습니다.

7. 비용의 자릿수 (C8)

논문에는 비용이 없습니다. 공개 설정(GPT-4o-mini, text-embedding-3-large, 청크 1,200 토큰, gleaning 1회, 질의 문맥 32,000 토큰)으로 **텍스트 처리만** 어림하면 다음과 같습니다. 가격(입력 $0.15/M, 출력 $0.60/M, 임베딩 $0.13/M)은 내 사전 지식이라 확인이 필요합니다.

벤치마크	문서 토큰	LLM 입력	LLM 출력	임베딩	합계(텍스트만)
DocBench (229문서, 1,102문항)	10.6 M	88.6 M	10.6 M	15.9 M	약 $22
MMLongBench (135문서, 1,082문항)	2.9 M	52.9 M	2.9 M	4.3 M	약 $10

여기에 그림·표·수식마다 부르는 VLM 호출(이미지 토큰 포함)이 더해집니다. 문서당 그림 수가 논문에 없어서 이 부분은 추정하지 않았습니다. 방법 하나를 벤치마크 하나에 돌리는 비용은 수십 달러 수준으로 보입니다. 질의 입력(문항당 약 33,500 토큰)이 의외로 큽니다. DocBench에서는 색인 추출 입력(약 48 M)과 비슷한 규모(약 37 M)이고, 문서가 짧은 MMLongBench에서는 색인(약 13 M)의 세 배쯤 됩니다.

04. 코드 검증: 논문 ↔ 구현 대조, 수치 재계산, 구현 가능성

- **대상:** `HKUDS/RAG-Anything` HEAD `2cd2b33` (2026-10-28, v1.4.4)와 논문 시점 커밋 `806ac9b` (2025-10-13; `config.py`, `modalprocessors.py`, `query.py`, `prompt.py`만 받아 비교). 기반 라이브러리 `HKUDS/LightRAG` HEAD `aaebfe1` (2026-10-14).
- **범위:** 코드 읽기와 CPU 재계산만 했습니다. 외부 API 호출, 패키지 설치, 모델 다운로드는 하지 않았습니다. 공개 저장소에는 논문 실험의 질의별 결과, 색인, 로그가 없어서 표를 결과 파일로 재계산할 수는 없습니다.

1. 모듈 매핑

논문	구현 위치	일치 여부
문서 분해, 모달 유형 (Eq. 1)	`parser.py` (MinerU 기본, Docling·PaddleOCR 선택), `config.py: parser="mineru"`	✅ 논문은 MinerU만 씀
모달별 처리기 (그림·표·수식)	`modalprocessors.py: ImageModalProcessor, TableModalProcessor, EquationModalProcessor, GenericModalProcessor`	✅ 오디오·비디오 처리기는 이후 추가(기본 꺼짐)
상세 설명 d^chunk + 개체 요약 e^entity	각 처리기의 `generate_description_only`, JSON `detailed_description` + `entity_info{entity_name, entity_type, summary}`	✅ 요약은 최대 100단어
문맥 창 C_j, δ	`ContextExtractor`, `config.py: context_window=1, context_mode="page", max_context_tokens=2000, context_filter_content_types=["text"]`	⚠️ D6
내부 개체 추출 R(d^chunk) (Eq. 2)	`_process_chunk_for_extraction` → LightRAG `extract_entities`	✅ LightRAG의 일반 추출 프롬프트 그대로
`belongs_to` 간선 (Eq. 3-4)	`modalprocessors.py` 857-887행: keywords `"belongs_to,part_of,contained_in"`, weight 10.0	✅ weight는 논문에 없음
두 그래프의 개체 정렬·병합	같은 `knowledge_graph_inst`에 `upsert_node`, LightRAG `merge_nodes_and_edges`	⚠️ D1
임베딩 표 T (Eq. 5)	LightRAG의 `entities_vdb`, `relationships_vdb`, `chunks_vdb`	✅ 그림 설명도 같은 chunks_vdb에 들어감
모달 인식 질의 인코딩, 융합 점수	해당 코드 없음	❌ D4
구조 탐색 + 의미 매칭	`query.py: aquery(mode="mix")` → LightRAG `kg_query` mix 모드	⚠️ D3
시각 자료 복원 + VLM 답 생성 (Eq. 6)	`aquery_vlm_enhanced`: 검색 프롬프트의 `Image Path:` 줄을 정규식으로 찾아 base64 이미지로 VLM에 전달	✅ 단, 재현 스크립트는 끔(D5)
실험 설정 (Sec. 3.1)	`reproduce/env.example`, `reproduce/query.py`	✅ D7
판정 프롬프트 (A.4, Fig. 10)	`reproduce/llm_answer_evaluator.py`	⚠️ D8
분석 프롬프트 (A.3, Fig. 7-9)	`prompt.py`	✅ 논문 시점과 일치, HEAD는 다름(D9)

2. 불일치와 논문에 적히지 않은 구현 요소

#	내용	영향
D1	**"두 그래프를 만든 뒤 개체 정렬로 병합"은 같은 그래프 저장소에 넣는 것입니다.** 교차 모달 쪽 노드와 텍스트 쪽 노드가 같은 LightRAG 그래프에 upsert되고, 이름 문자열이 정확히 같은 노드만 합쳐집니다. 별도의 정렬 단계(동의어, 표기 변형, 임베딩 유사도)는 없습니다. 앵커 이름에는 `" (image)"` 같은 유형 접미사가 붙으므로(HEAD 1139-1141행, 논문 시점도 같음) 앵커 자체는 절대 병합되지 않습니다. Fig. 1의 "Text VDB"와 "Multi-modal VDB"도 실제로는 하나의 벡터 DB입니다.	두 그래프는 내부 개체 이름이 우연히 같을 때만 이어집니다(`03` §5). "개체 정렬"의 기여를 따로 잰 절제는 없습니다.
D2	**사례 연구가 설명하는 표·그림 내부 구조 그래프가 코드에 없습니다.** 행 머리글, 열 머리글, 셀, 단위 노드와 row-of, column-of, header-applies-to, unit-of 간선(Sec. 3.4), 패널·축·범례 노드(Fig. 3 설명), bar-of, label-applies-to 간선(A.2)을 코드 전체에서 검색해도 찾을 수 없습니다. 표는 MinerU가 준 HTML 본문과 LLM 분석문을 이어 붙인 **청크 하나**(`table_chunk` 프롬프트)로 저장되고, 그 분석문에서 일반 개체 추출을 돌릴 뿐입니다.	사례 연구의 "왜 맞혔는가" 설명은 코드가 하는 일이 아닙니다. LLM 추출이 우연히 그런 개체를 낼 수는 있지만, 스키마로 보장되지 않습니다. 셀 위치를 맞히는 일은 실제로는 reader가 HTML 표를 읽으며 합니다.
D3	**"구조 탐색"은 LightRAG의 키워드 벡터 검색 + 1-hop입니다.** 논문은 "질의어와 개체의 정확한 매칭 후 지정 hop까지 이웃 확장"이라고 씁니다. 코드에서는 LLM이 질의에서 저수준·고수준 키워드를 뽑고, 개체·관계 벡터 DB를 코사인 유사도로 검색합니다(LightRAG `_get_node_data`, `top_k` 기본 40, 재현 설정 50, 임계값 0.2). 그다음 검색된 개체의 1-hop 간선만 펼칩니다. hop 수를 정하는 설정은 없습니다.	"정확한 매칭"이 아니라 임베딩 매칭입니다. 논문의 두 경로(구조/의미)는 코드에서 둘 다 벡터 검색에서 출발합니다.
D4	**모달 인식 질의 인코딩과 다중 신호 융합 점수가 없습니다.** 질의에서 "figure", "table" 같은 단어를 찾아 모달 선호를 정하는 코드, 그리고 구조 중요도·의미 유사도·모달 선호를 합치는 점수가 `query.py`(논문 시점, HEAD 모두)에 없습니다. 후보는 LightRAG mix 모드가 개체·관계·벡터 청크를 토큰 예산 안에서 합치고, 선택적으로 reranker가 다시 정렬합니다.	논문이 Sec. 2.3에서 서술한 핵심 검색 장치 두 개가 공개 코드로 재현되지 않습니다. 부록 A.5가 말한 "텍스트 중심 검색 편향"은 모달 선호 신호가 없는 코드 구조와 잘 맞습니다.
D5	**재현 스크립트는 VLM 답 생성을 끕니다.** `reproduce/query.py`가 `rag.aquery(..., mode="mix", response_type="One Sentence", vlm_enhanced=False)`로 질의합니다. 그러면 검색 문맥(그림의 텍스트 설명 포함)만 LLM에 들어가고, Eq. 6의 원본 이미지 복원은 일어나지 않습니다. 재현 스크립트는 논문 6개월 뒤인 2026-04-13(`a9fb942`)에 추가되었습니다.	논문 수치가 VLM 답 생성으로 나온 것인지 알 수 없습니다. 공개 스크립트로 재현하면 Eq. 6이 빠진 설정이 됩니다.
D6	문맥 창 δ는 **쪽 단위**가 기본입니다(`context_mode="page"`, `context_window=1` → 앞뒤 1쪽). 텍스트 항목만 문맥으로 쓰고(`context_filter_content_types=["text"]`) 2,000 토큰에서 자릅니다. 논문은 δ를 단위 인덱스 거리	k – j
D7	실험 설정은 재현 파일과 맞습니다. `reproduce/env.example`은 GPT-4o-mini(OpenRouter), text-embedding-3-large(3,072), bge-reranker-v2-m3, `MAX_ENTITY_TOKENS=12000` + `MAX_RELATION_TOKENS=8000` = 20,000(논문 "개체+관계 20,000")이고, `MAX_TOTAL_TOKENS=32000`에서 20,000을 빼면 청크 12,000(논문 "청크 12,000")입니다. `TOP_K=50`, `CHUNK_TOP_K=10`, mix 모드, 한 문장 답도 같습니다.	✅ 다만 LightRAG 버전은 논문 시점에 고정되지 않았습니다(`lightrag-hku` 버전 미지정).
D8	**판정 스크립트에 두 가지 문제가 있습니다.** (a) 기본 판정 모드가 `comprehensive`로, 논문 Fig. 10의 정확도 전용 프롬프트가 아니라 여러 지표(관련성, 완결성, 충실성 등)를 함께 요구합니다. (b) JSON 파싱 실패 시 fallback이 긍정 키워드를 부분 문자열로 먼저 검사해서 "incorrect", "mismatch", "not accurate"가 정답(1)으로 채점됩니다(`03` §6; toy 9개 중 4개 오채점). 판정 모델은 GPT-4o-mini, temperature 0.1입니다.	(b)는 실패 빈도만큼 정확도를 부풀립니다. 빈도는 공개되지 않았습니다. 고치는 법은 간단합니다(부정 키워드를 먼저, 단어 경계로 검사하거나 구조화 출력 강제).
D9	논문 부록의 분석 프롬프트(Fig. 7-9)는 논문 시점 `806ac9b`의 `prompt.py`와 같습니다. HEAD는 2026-05-22(섹션 경로, 주변 텍스트 메타데이터)와 2026-08-13(다이어그램 노드·간선을 모두 열거하라는 지시, 파일명 대신 의미 있는 개체명)에 바뀌었습니다.	HEAD로 돌리면 논문과 다른 설정입니다. 논문 재현에는 `806ac9b`나 v1.2.8(2025-09-22)을 써야 합니다.

#	내용	영향
D10	앵커의 내부 개체 추출은 LightRAG의 gleaning(기본 1회 추가 추출)을 그대로 씁니다. 비텍스트 단위 하나당 VLM 설명 호출 1회 + 추출 호출 2회가 듭니다.	논문에 비용 보고가 없습니다(`03` §7).

3. 수치 재계산 (`analysis/table_check.py`, `analysis/code_logic_toy.py`)

#	확인 항목 (출처)	재계산	판정
C1	Overall = 영역 정확도의 문항 가중 평균 (Table 2-5)	11개 행 중 9개가 0.1%p 안에서 일치. 나머지는 **MMGraphRAG(Table 2) 61.0 vs 56.7**, MMLongBench GPT-4o-mini 33.5 vs 34.05	⚠ MMGraphRAG 행 불일치
C1b	단일 칸 오기 가설	Law 40.0 → 64.7 또는 Fin 52.8 → 69.2면 61.0. Fig. 2 가중 평균 60.9가 Overall 61.0을 지지	가설
C2	유형 3개(Txt, Mm, Una)로 Overall 설명	7개 행 공통 비율 맞춤의 잔차 최대 2.7%p	표에 없는 유형이 있을 가능성
C3	차이의 표본 오차	DocBench vs MMGraphRAG +2.4 (p ≈ 0.25), vs LightRAG +5.0 (p ≈ 0.016). MMLongBench vs MMGraphRAG +5.1 (p ≈ 0.016), vs LightRAG +3.9 (p ≈ 0.065). Chunk-only –3.4 (p ≈ 0.10)	비대응 근사
C4	평균 쪽수 (Table 1 vs 5·6)	DocBench 66 vs 61.0 (반올림 허용 60.5-61.5 밖), MMLongBench 47.5 vs 48.3 (47.8-48.8 밖)	⚠ 불일치
C5	Fig. 2 (픽셀 판독)	본문 길이별 수치와 0.1%p 안에서 일치. 문항 수 합 DocBench 약 1,079, MMLongBench 약 971(표 1,082). MMLongBench 200쪽 이상 n = 5(3 vs 2), 1-10쪽 n = 16(3 vs 2)	극단 구간은 근거 아님
C6	GPT-4o-mini 기준선의 50쪽 제한	DocBench 금융(평균 192쪽) 약 26%, 정부 72%만 봄	기준선이 구조적으로 불리
C7	이득의 출처	LightRAG 대비 Txt 0.0, Mm +16.6, Una –0.8. MMGraphRAG 대비 Una –14.5	이득은 Mm, 손해는 Una
C8	비용 (텍스트 처리만, 가격은 가정)	DocBench 약 $22, MMLongBench 약 $10 / 방법 1회	논문에 없음
C9	그림 정합성	Fig. 12의 질문이 Fig. 3 질문의 복사본	편집 오류
E	판정 fallback	toy 9개 중 4개 오채점	D8
G	앵커-텍스트 그래프 연결	P = 1 – (1 – p)^m	`03` §5

4. 구현 가능성 판단

4.1 공식 코드를 그대로 쓰는 경우 (도구로 사용)

항목	요구 사항	비고
설치	`pip install raganything` (+ MinerU 또는 Docling). Python 패키지라 macOS에서도 설치될 것으로 보임	이 맥에서는 설치하지 않았습니다. MinerU의 Apple Silicon 가속(MPS) 지원 여부는 확인하지 않았습니다
모델	LLM, VLM, 임베딩 함수 세 개를 사용자가 넘김. 논문 설정은 GPT-4o-mini + text-embedding-3-large + bge-reranker-v2-m3(API)	저장소에 Ollama, LM Studio, vLLM 연동 예제가 있어 로컬 모델로도 돌릴 수 있습니다

항목	요구 사항	비고
GPU	필수 아님. API 모델이면 CPU만으로 됨. 로컬 모델이면 이 맥의 Apple GPU(Metal, 통합 메모리 약 24 GiB)로 7-14B급 LLM·VLM이 현실적	CUDA 전용 요소는 vLLM 예제뿐입니다
비용	벤치마크 하나에 방법 하나를 돌리면 텍스트 처리만 수십 달러(C8). 이미지 처리 비용 별도	내 문서 몇십 개면 몇 달러 수준으로 추정
데이터	DocBench, MMLongBench-Doc는 공개 데이터셋	각 라이선스 확인 필요
프라이버시	API를 쓰면 문서 전체(텍스트와 그림)가 외부 서버로 갑니다	개인 vault나 비공개 PDF에는 로컬 모델이 맞습니다

판단: 도구로 쓰는 것은 **가능하고 장벽이 낮습니다.** MIT 라이선스이고, 설치가 pip 하나이며, 이 맥에서 API 모델로도 로컬 모델로도 돌릴 수 있습니다. 다만 이번에는 설치와 API 호출을 하지 않았으므로(재현 보류) 실제로 도는지는 확인하지 않았습니다.

4.2 논문 수치를 재현하는 경우

걸림돌	내용
버전	논문 시점 코드 `806ac9b` (또는 v1.2.8)와 그 당시 LightRAG 버전을 맞춰야 합니다. LightRAG 버전은 기록이 없습니다
설정 차이	공개 재현 스크립트는 VLM 답 생성을 끕니다(D5). 논문이 켰는지 알 수 없습니다
없는 장치	모달 선호와 융합 점수(D4), 표·그림 구조 그래프(D2)는 코드에 없어서, 논문 설명대로의 시스템은 재현할 수 없습니다
판정	판정 버그(D8)를 고치고 정확도 전용 프롬프트를 쓸지 정해야 합니다. 판정 모델이 같으면 결과가 판정자에 따라 달라질 수 있습니다
기준선	MMGraphRAG 결과(특히 Table 2의 불일치 행, C1)를 확인하려면 기준선도 다시 돌려야 합니다
비용·시간	방법 4개 × 벤치마크 2개면 텍스트 처리만 약 $100-150, VLM 처리와 재시도를 더하면 그 이상입니다. 시간은 API 속도 제한에 달려 있습니다

판단: 코드와 설정 파일이 있어서 "대략 비슷한 숫자"를 다시 내는 것은 비용만 내면 가능합니다. 하지만 논문 Sec. 2.3이 설명한 시스템 그대로 재현할 수는 없습니다. 공개 코드는 논문보다 단순한, LightRAG mix 모드 + 멀티모달 앵커 시스템입니다.

4.3 아이디어만 다시 구현하는 경우

구성 요소	난이도	메모
비텍스트 단위 → VLM 설명 + 앵커 노드 + belongs_to 간선	낮음	처리기 하나가 프롬프트 + JSON 파싱 + upsert 몇 번입니다. LightRAG 위에 수백 줄
검색 이미지의 원본 복원 + VLM 답	낮음	청크에 이미지 경로를 남기고 정규식으로 되찾는 방식이 단순하고 실용적입니다
논문이 서술했지만 코드에 없는 모달 선호·융합 점수, 표 셀 그래프	중간	직접 설계해야 합니다. 효과가 검증되지 않은 부분이라 연구 질문이 됩니다
평가	중간	판정자 편향과 파싱 실패 처리를 제대로 하려면 구조화 출력과 사람 검수 표본이 필요합니다

요약: RAG-Anything은 "멀티모달 문서를 LightRAG 그래프에 넣는 실용 도구"로서는 바로 쓸 수 있습니다. 하지만 논문의 검색 설계(모달 인식, 융합 점수, 구조 그래프)는 코드로 뒷받침되지 않습니다. 성능 주장은 표 정합성 문제(C1, C4)와 작은 차이(C3)를 감안해서 읽어야 합니다.

02. 선행연구와 계보, 관련 GitHub

논문의 참고문헌에는 번호가 없어서 저자·연도로 적고, arXiv ID는 참고문헌 목록에 적힌 값을 옮겼습니다. 코드와 GitHub 정보는 2026-10-28에 `gh api`로 직접 확인했습니다. 그 밖의 논문은 따로 열어 검증하지 않았습니다.

1. 계보도

```
[텍스트 RAG]
  Graph RAG 계열
    GraphRAG (Edge et al., 2024) arXiv:2404.16130   커뮤니티 요약 기반 질의 중심 요약
      └ LightRAG (Guo et al., 2024) arXiv:2410.05779   ← 같은 그룹(HKUDS). 개체·관계 그래프 + 이중
수준(low/high) 키워드 검색
    HippoRAG (NeurIPS 2024), GNN-RAG arXiv:2405.20139, RAPTOR (ICLR 2024), ArchRAG
arXiv:2502.09891
        |
[멀티모달 RAG]
  MM-VID arXiv:2310.19773 (영상 → 텍스트), VideoRAG arXiv:2502.01549 (같은 그룹)
  VisRAG arXiv:2410.10594 (문서를 이미지 그대로 검색, 파싱 없음)
  MMGraphRAG arXiv:2507.20804 (장면 그래프 + 텍스트 그래프, 표·수식은 텍스트 취급)
        |
  ★ RAG-Anything (2025-10): MinerU로 파싱 → 비텍스트 단위를 VLM 설명 + 앵커 노드로 LightRAG 그래프에 삽입
```

차별점 요약: 구성 요소는 대부분 같은 그룹의 LightRAG에서 옵니다(개체 추출, 그래프 저장, 키워드 이중 검색, `mix` 모드). 새로운 부분은 세 가지입니다. 비텍스트 단위를 모달별 프롬프트로 설명하는 처리기, 설명문에서 뽑은 개체를 앵커로 묶는 `belongs_to` 간선, 검색된 이미지 경로를 원본 이미지로 되돌려 VLM에 넣는 답 생성입니다. 비교 대상 중 가장 가까운 것은 MMGraphRAG인데, 저자들은 MMGraphRAG가 표와 수식을 개체로 다루지 않는다는 점을 차이로 듭니다.

2. 구성 요소별 출처

역할	출처	이 논문에서의 쓰임
문서 파싱	MinerU (Wang et al., 2024) arXiv:2409.18839	텍스트·이미지·표·수식 추출. 코드 기본 파서이고 Docling, PaddleOCR도 선택 가능
텍스트 그래프·검색	LightRAG (Guo et al., 2024)	개체·관계 추출 프롬프트, 그래프·벡터 저장소, `local/global/hybrid/mix/naive` 질의 모드
그래프 RAG 원형	GraphRAG (Edge et al., 2024)	텍스트 그래프 구성 방식의 원형으로 인용
비교 기준선	MMGraphRAG (Wan & Yu, 2025)	같은 백본으로 비교
임베딩·reranker	text-embedding-3-large, bge-reranker-v2-m3	실험 설정
벤치마크	DocBench (Zou et al., 2024) arXiv:2407.10701, MMLongBench-Doc (Ma et al., NeurIPS 2024)	평가
판정	GPT-4o-mini (부록 A.4)	0/1 정확도

3. 이 저장소의 다른 논문과의 관계

이 저장소의 논문들(로봇 학습, 제어)과 방법상 직접 연결은 없습니다. 연결점은 연구 도구 쪽입니다.

대상	관계
이 저장소 자체 (`research_roadmap`)	논문 PDF, 그림이 있는 쪽 이미지, 표 재계산 문서가 쌓이는 코퍼스입니다. "그림 몇 번이 무엇을 보여 주나", "Table 2의 어느 칸" 같은 질의를 다루려면 이 논문 같은 멀티모달 문서 RAG가 필요합니다. 다만 이 저장소의 분석 습관(표 가중 평균 검산, 문항 수 확인)을 이 논문에 적용하면 MMGraphRAG Overall 불일치 같은 문제가 바로 드러납니다(`04` C1).
ZeroDex (`zerodex/`)	둘 다 VLM에 그림을 보여 주고 구조화된 출력(JSON)을 받아 쓰는 파이프라인입니다. VLM 출력 파싱이 실패할 때의 처리(이 논문 코드의 fallback 버그, `04` D8)는 두 경우 모두 결과 신뢰도에 직접 영향을 줍니다.

4. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

주제	관련성	근거 수준
ColPali / ColQwen (문서 쪽 이미지를 late-interaction으로 임베딩)	파싱 없이 쪽 이미지를 바로 검색하는 강력한 대안입니다. VisRAG와 같은 방향이고, MinerU 파싱 오류(부록 A.5의 병합 셀 실패)를 우회합니다. 비교 기준선에 없습니다.	내 사전 지식. 서지 확인 필요
M3DocRAG (다쪽 문서 멀티모달 RAG)	같은 과제(MMLongBench-Doc 포함)를 다룬 직접 경쟁 방법으로 보입니다.	내 사전 지식. 서지·수치 확인 필요
RAG (Lewis et al., 2020)	검색 증강 생성의 원형입니다. 논문은 서베이만 인용합니다.	내 사전 지식
LLM-as-a-judge 편향 연구	생성 모델과 판정 모델이 같을 때의 자기 선호 편향은 알려진 문제입니다(`01` §4.25).	내 해석

한계: 4절의 서지 정보는 내 사전 지식에 근거합니다. 블로그나 git에 올리기 전에 원문을 직접 확인해야 합니다.

5. 관련 GitHub 저장소 (2026-10-28 확인)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인한 커밋
HKUDS/RAG-Anything	이 논문의 구현. `raganything/` (처리기, 질의), `reproduce/` (색인·질의·판정 스크립트, 2026-04-13 추가)	**공식** (논문 초록에 링크)	MIT	24,244 (fork 2,837)	2026-10-28	HEAD `2cd2b33` (v1.4.4), 논문 시점 `806ac9b` (2025-10-13). 가장 가까운 릴리스 v1.2.8(2025-09-22)
HKUDS/LightRAG	기반 라이브러리. 그래프 추출, 저장소, 질의 모드	같은 그룹의 공식 저장소	MIT	40,388	2026-10-27	`aaebfe1` (shallow clone)
opendatalab/MinerU	기본 문서 파서	외부 의존성	Apache-2.0 + 추가 조건(상업적 사용 임계값)	82,671	2026-10-28	`b7f3dd0` (clone 안 함)
docling-project/docling	대체 파서	외부 의존성	MIT	69,738	2026-10-28	`9051777` (clone 안 함)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인한 커밋
wanxueyao/MMGraphRAG	비교 기준선 MMGraphRAG	논문 저자 저장소로 보임	Apache-2.0	94	2026-03-10	`ec0065d` (clone 안 함)
OpenBMB/VisRAG	파싱 없는 비전 기반 RAG (관련 연구)	VisRAG 공식	Apache-2.0	986	2026-08-09	`7205787` (clone 안 함)
microsoft/graphrag	GraphRAG 원형	공식	MIT	36,473	2026-10-27	`c600ff8` (clone 안 함)

- 실제로 코드를 읽은 것은 RAG-Anything(HEAD와 논문 시점 파일 4개)과 LightRAG(HEAD)입니다. 나머지는 메타데이터만 확인했습니다.
- 논문 시점에는 `lightrag-hku` 버전이 고정되어 있지 않았습니다(`requirements.txt`). 지금은 `lightrag-hku<1.5` 입니다. 논문 결과를 재현하려면 2025-10 당시의 LightRAG 버전을 따로 맞춰야 합니다.
- MinerU 라이선스는 현재 Apache-2.0에 상업적 사용 조건을 덧붙인 형태입니다. 연구 사용에는 문제가 없어 보이지만, 상업적으로 쓸 때는 조건을 확인해야 합니다.